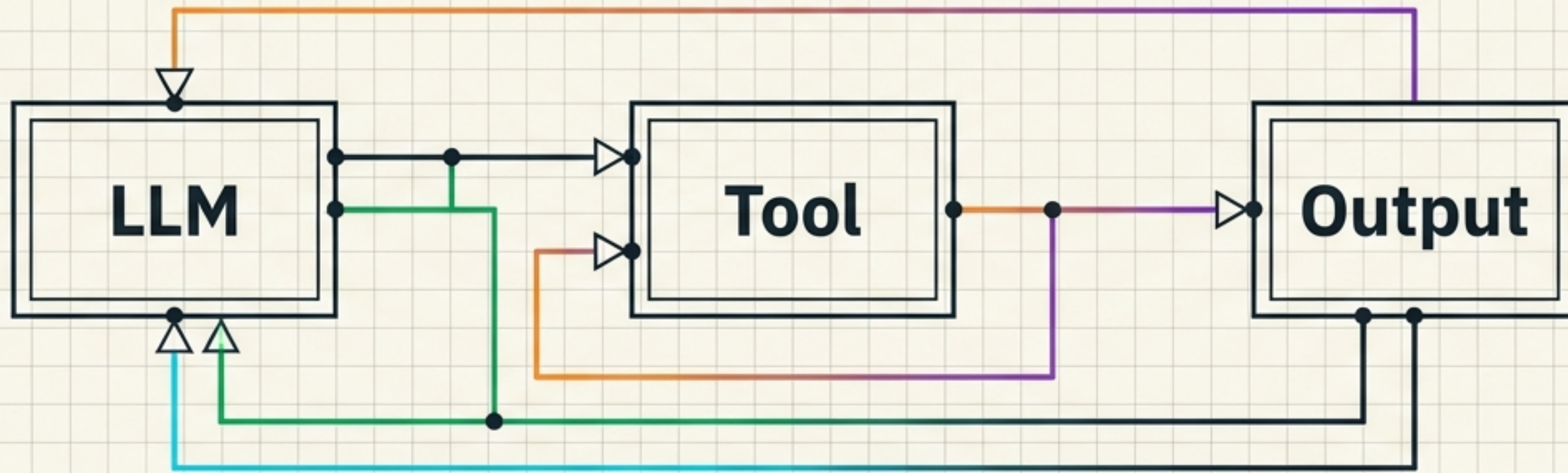


Grep คือคำตอบสุดท้ายหรือไม่?

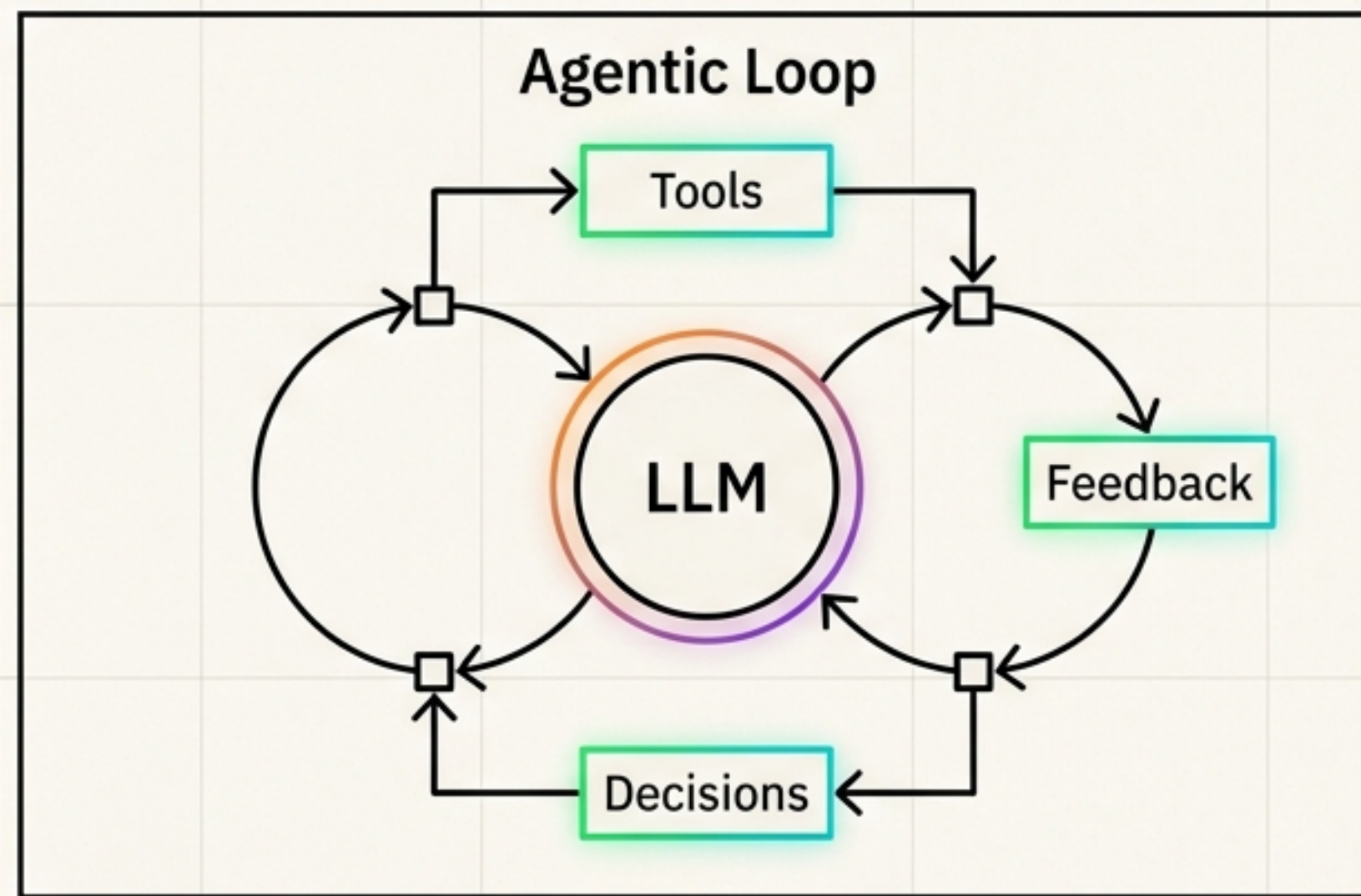
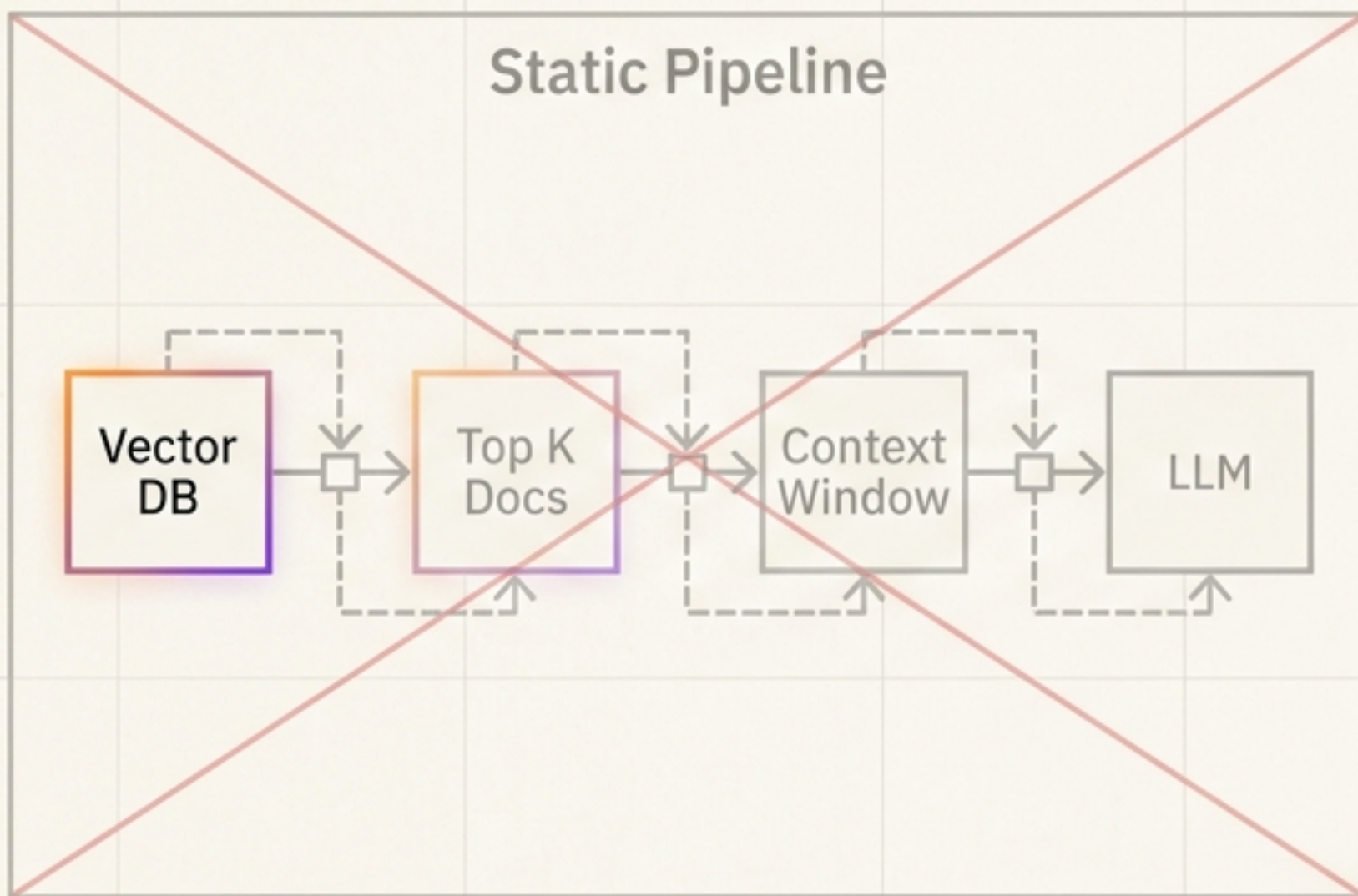
สถาปัตยกรรมของ Agent เปลี่ยนแปลงประสิทธิภาพ
การค้นหาข้อมูล (Agentic Search) อย่างไร



ถอดรหัสงานวิจัยจาก 'Is Grep All You Need?' (Sen et al.)

จุดบอดของการทำ RAG ในปัจจุบัน

วงการ AI ยึดติดว่า Vector Search คือมาตรฐาน (Default) แต่ RAG สำหรับ Autonomous Agents ไม่ใช่แค่เส้นทางข้อมูลทางเดียว (Static Pipeline)



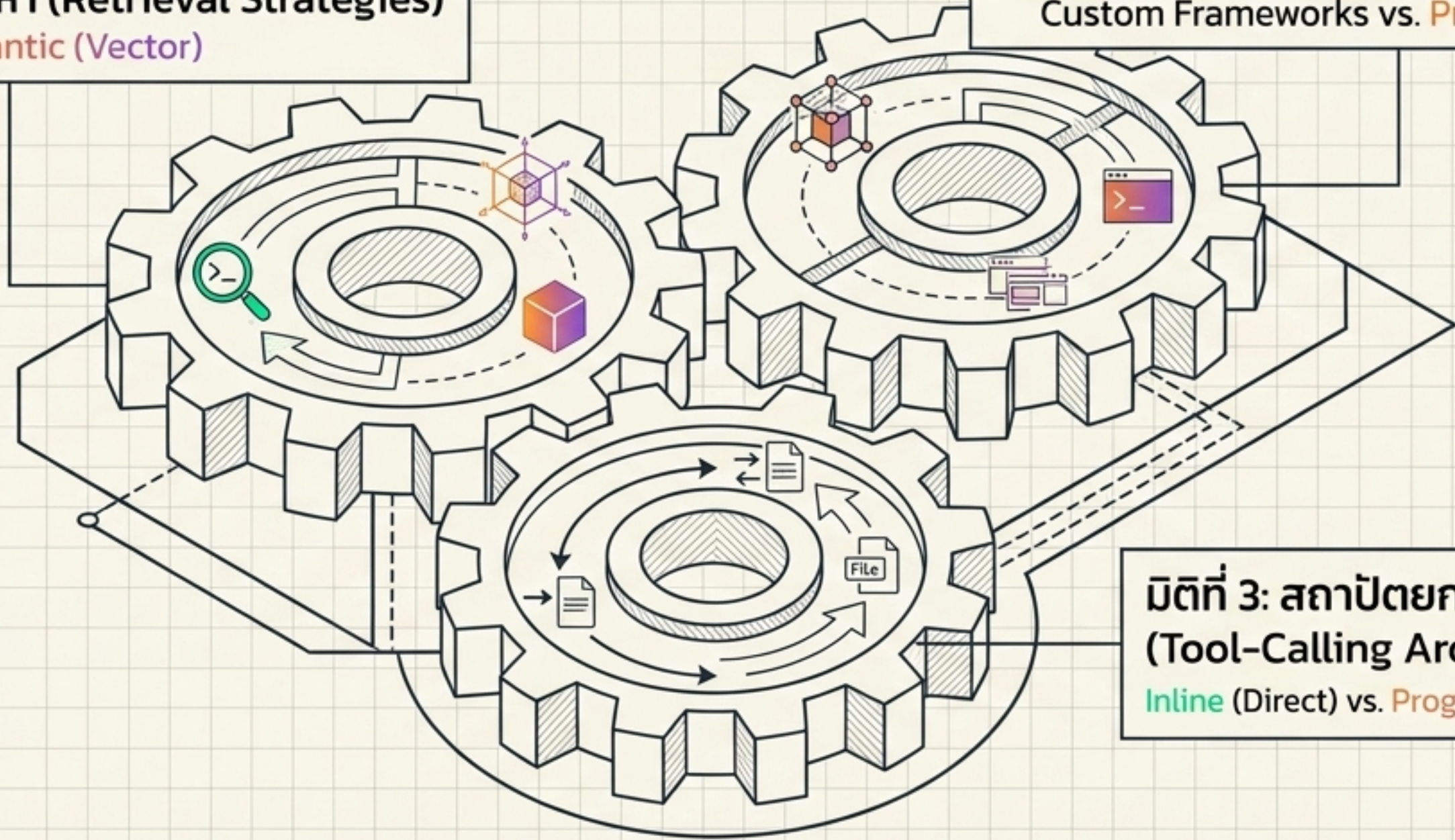
ประสิทธิภาพของการค้นหาไม่ได้ขึ้นอยู่กับ "ดัชนี" (Index) เพียงอย่างเดียว แต่ถูกกำหนดโดยกระบวนการเรียกใช้เครื่องมือแบบวนซ้ำ (Iterative Tool-calling) ของ Agent

3 มิติของระบบ Agentic Search

มิติที่ 1: กลยุทธ์การค้นหา (Retrieval Strategies)
Lexical (Grep) vs. Semantic (Vector)

มิติที่ 2: ตัวควบคุม Agent (Agent Harnesses)
Custom Frameworks vs. Provider-Native CLIs

มิติที่ 3: สถาปัตยกรรมการส่งข้อมูล
(Tool-Calling Architecture)
Inline (Direct) vs. Programmatic (File-Based)

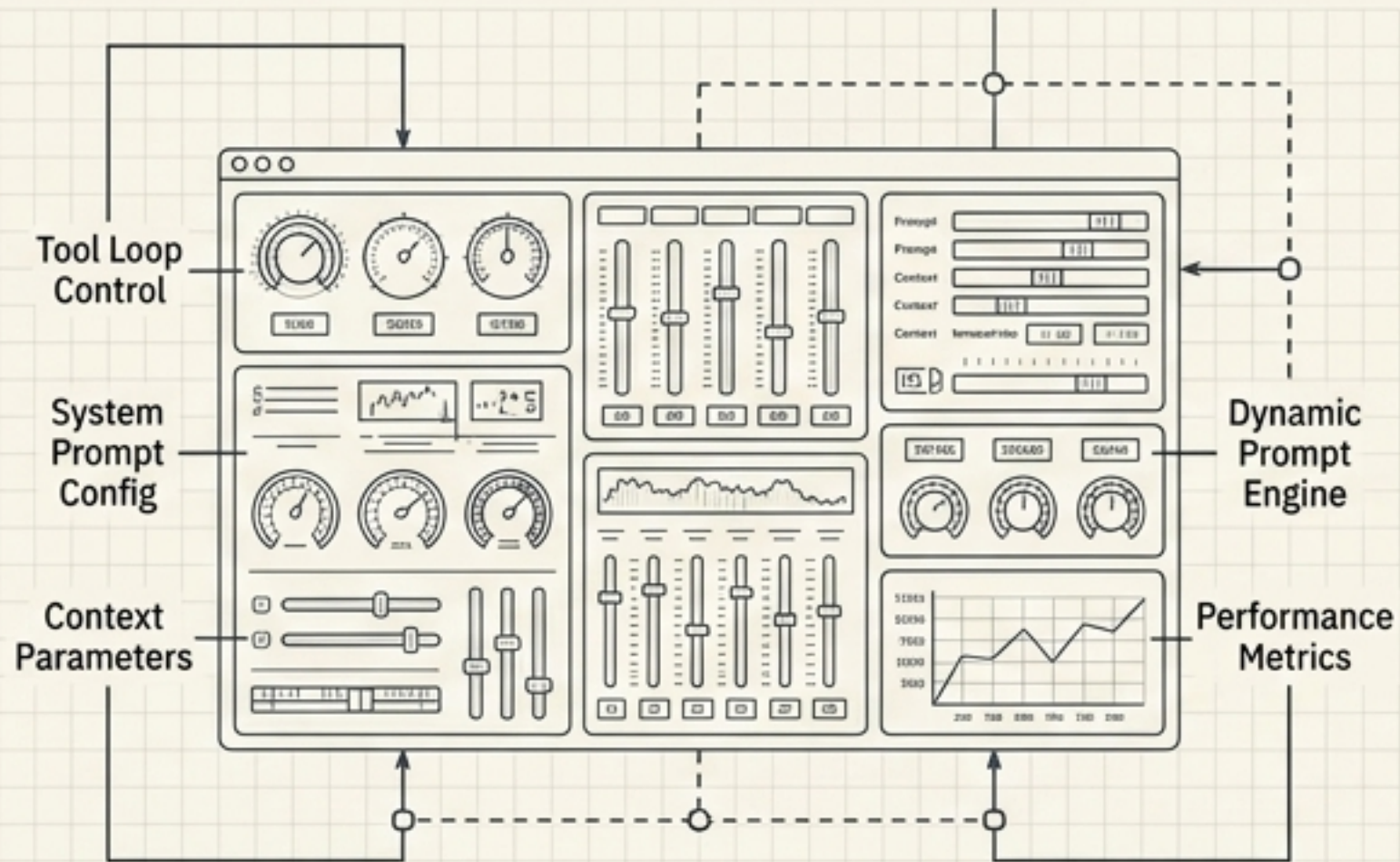


“คุณไม่สามารถประเมินระบบถึงข้อมูลได้ หากไม่พิจารณาชั้น Orchestration”

มิตที่ 1: เปรียบเทียบกระบวนการค้นหา

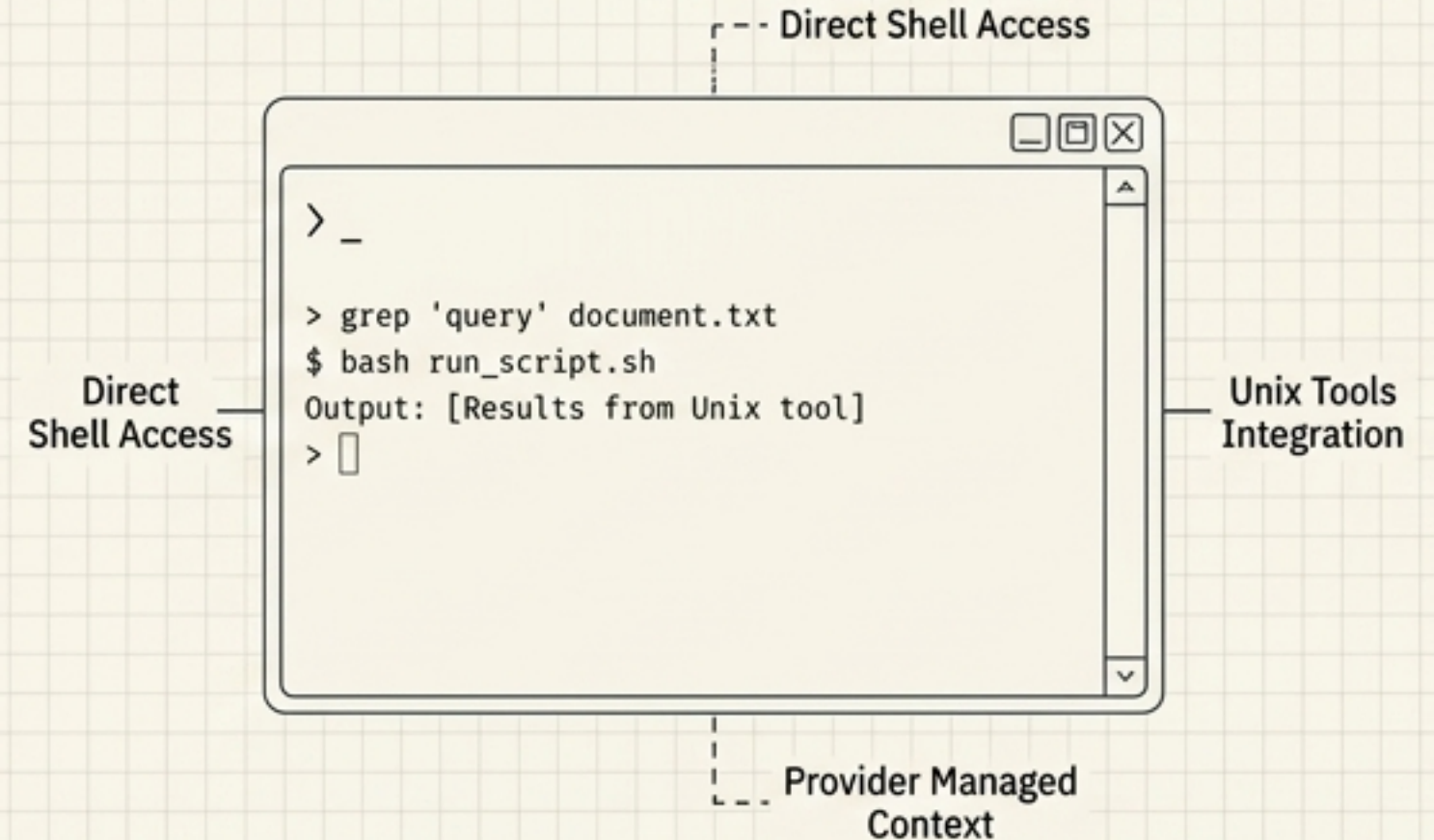
	Grep (Lexical)	Vector (Semantic)
กลไก (Mechanism)	จับคู่รูปแบบข้อความที่แม่นยำ (Exact/Regex matching)	ฝังข้อมูลในพื้นที่เวกเตอร์ (Dense embedding / ANN)
จุดแข็ง (Strengths)	ต้นทุนต่ำ, ค้นหาข้อมูลที่เป็นตัวอักษร/ตัวเลขเป๊ะๆ ได้ดี (Literal spans)	เข้าใจบริบทเชิงความหมาย (Semantics), รองรับการถอดความ (Paraphrasing)
ความเสี่ยงใน โหมด Agent (Agentic Risk)	หาก Agent ใช้คำค้นหาผิด จะไม่ได้ผลลัพธ์เลย (Vocabulary mismatch)	ดึงข้อมูลที่ "ความหมายใกล้เคียง" แต่ผิดบริบทมาหลอก LLM (Topical false friends)

มิตีที่ 2: ตัวควบคุม (Agent Harnesses)



Custom Harness (Chronos)

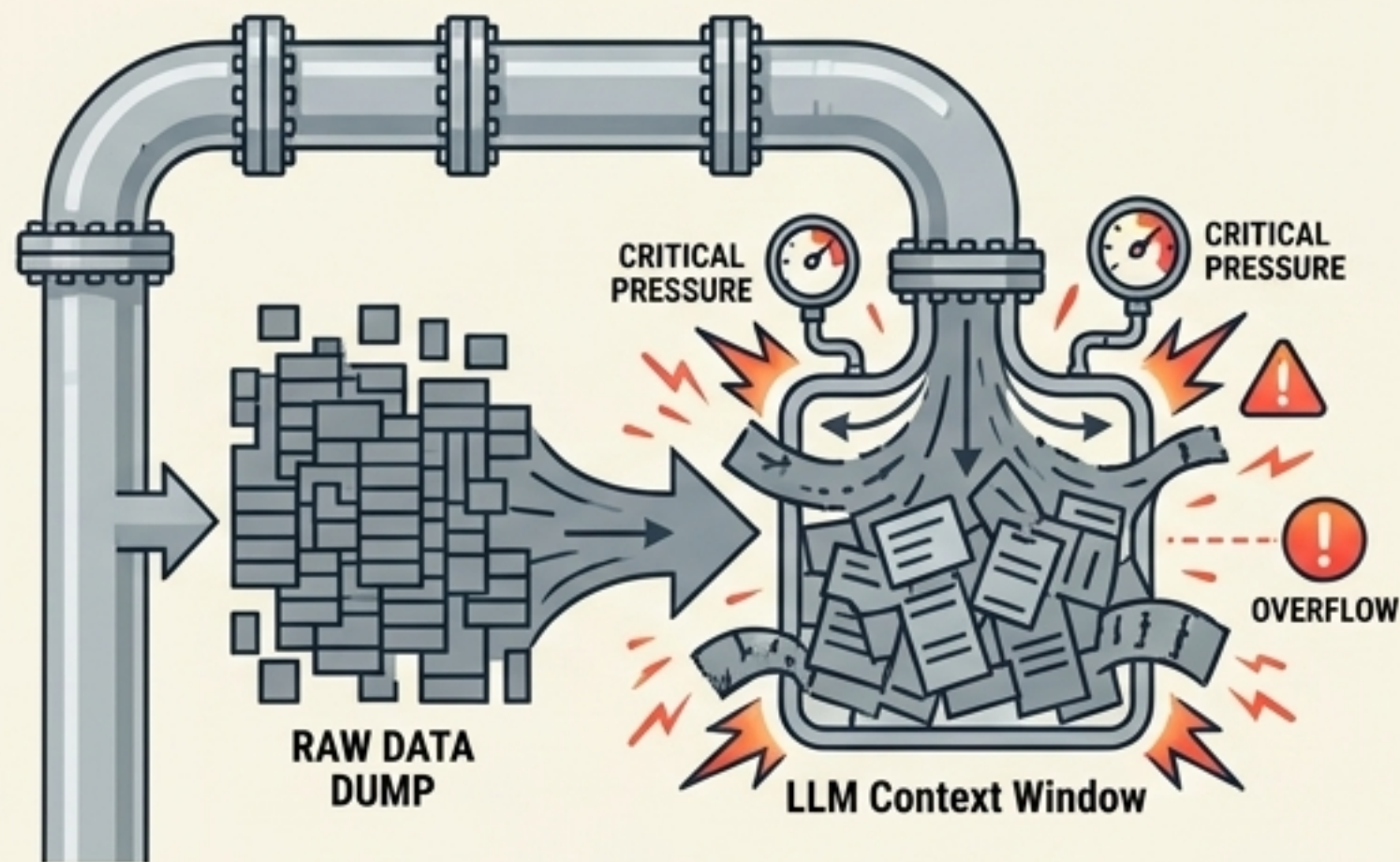
- **คุณสมบัติ:** ควบคุม Tool loop, System Prompt และ Context ได้ อย่างละเอียด
- **ข้อดี:** สามารถปรับแต่ง Prompt แบบไดนามิก (Dynamic prompting)
- **ข้อเสีย:** ต้นทุนการพัฒนาระบบ (Engineering overhead) สูง



Provider-Native CLI (Claude Code/Codex)

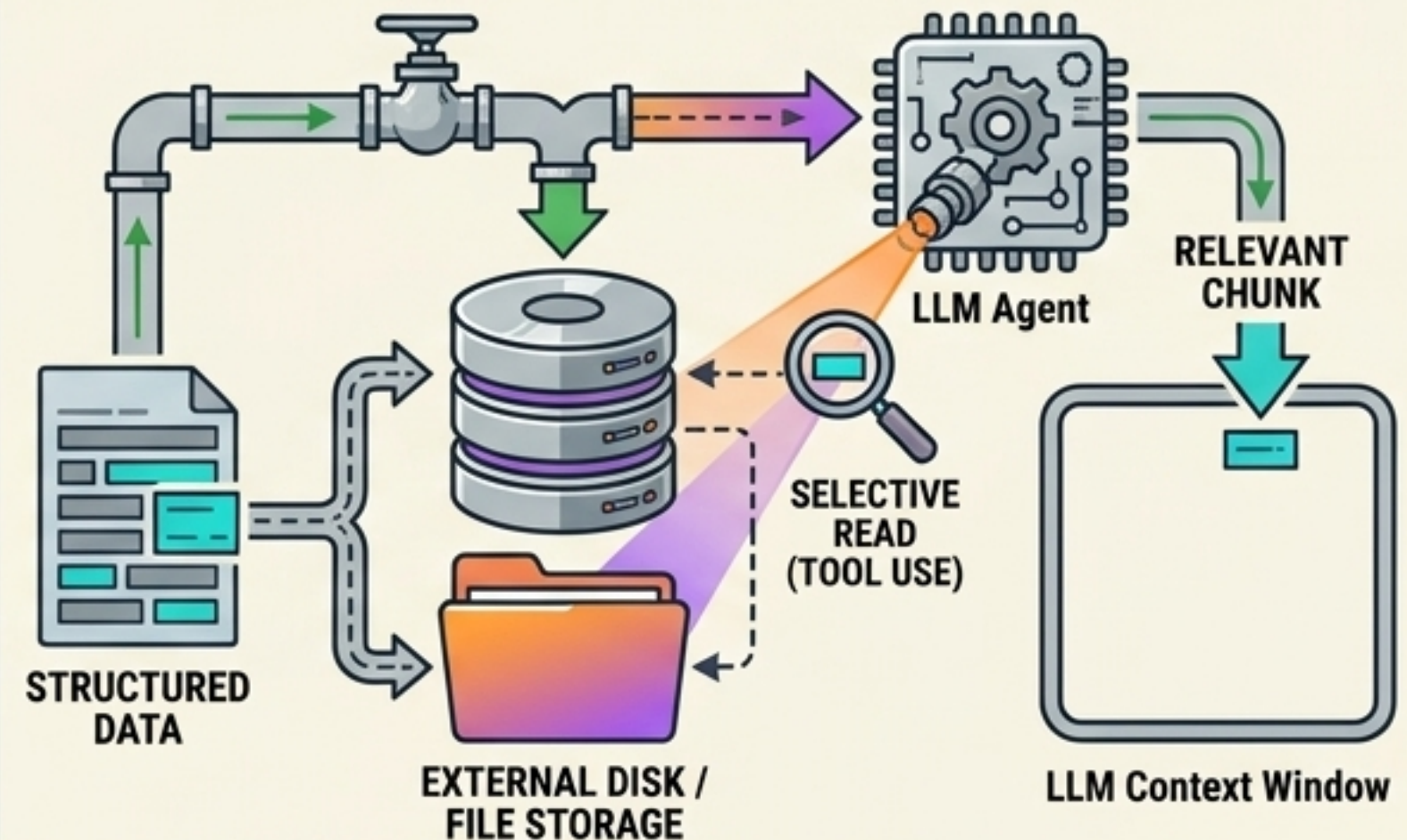
- **คุณสมบัติ:** ให้ Agent เข้าถึง Unix tools (เช่น grep, bash) ได้โดยตรงผ่าน Shell
- **ข้อดี:** ใช้งานง่าย อาศัยการปรับแต่ง Context จากผู้ให้บริการโดยตรง
- **ข้อเสีย:** ขาดความยืดหยุ่น การทำงานภายในเป็นกล่องดำ (Opaque)

มิติดที่ 3: กับดักของบริบท (The Context Trap)



Standard (Inline)

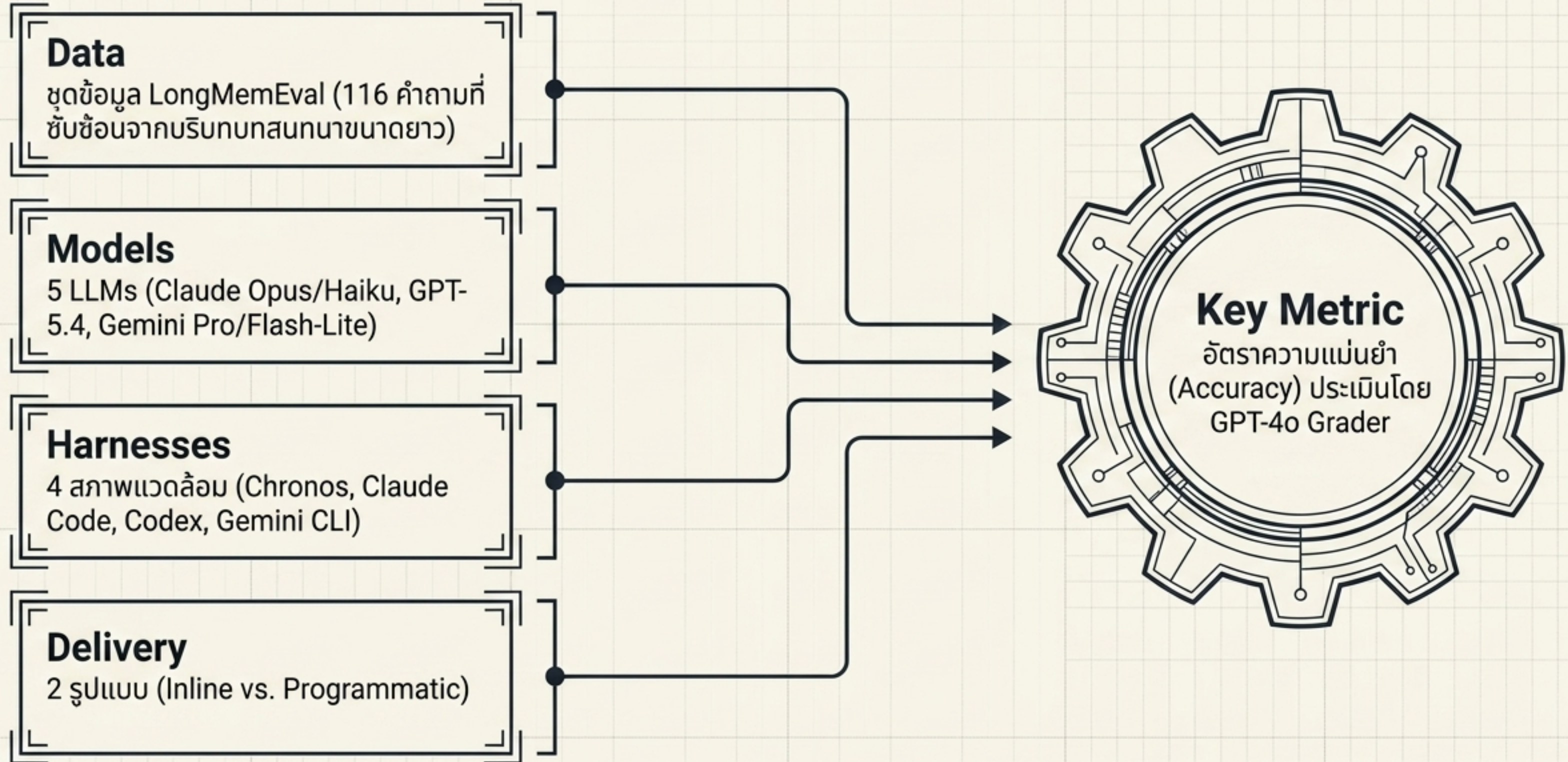
- กลไก: ส่งผลลัพธ์การค้นหาเข้า Context Window โดยตรง
- ผลกระทบ: ใช้งานง่าย แต่สร้าง "Context Pressure" แหย่งพื้นที่ความจำหลัก



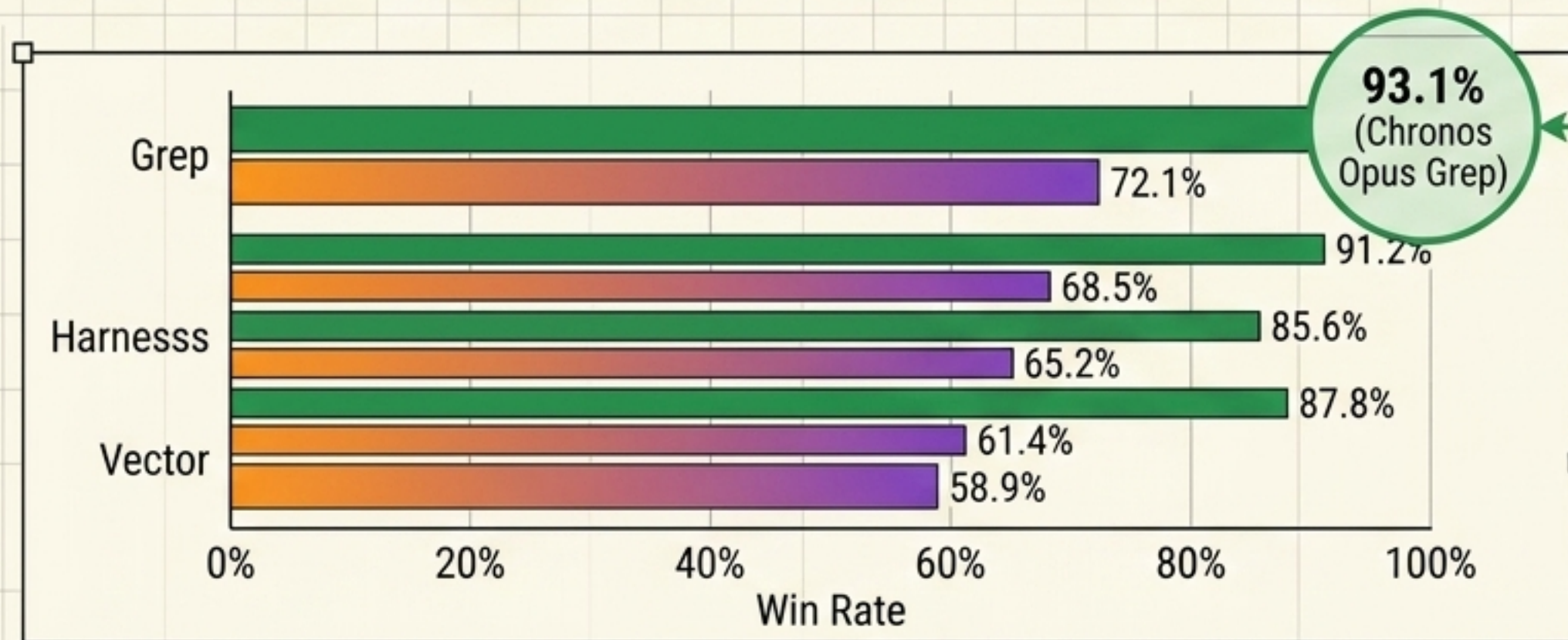
Programmatic (File-Based)

- กลไก: เขียนผลลัพธ์ลงไฟล์ (Disk) แล้วให้ Agent สั่งอ่านเมื่อต้องการ
- ผลกระทบ: ประหยัด Token แต่เพิ่มความซับซ้อน Agent ต้องมีทักษะการใช้ Tool ที่สูงขึ้น (Read -> Integrate -> Retry)

สังเวียนทดสอบที่ 1: การประเมินผลแบบ End-to-End



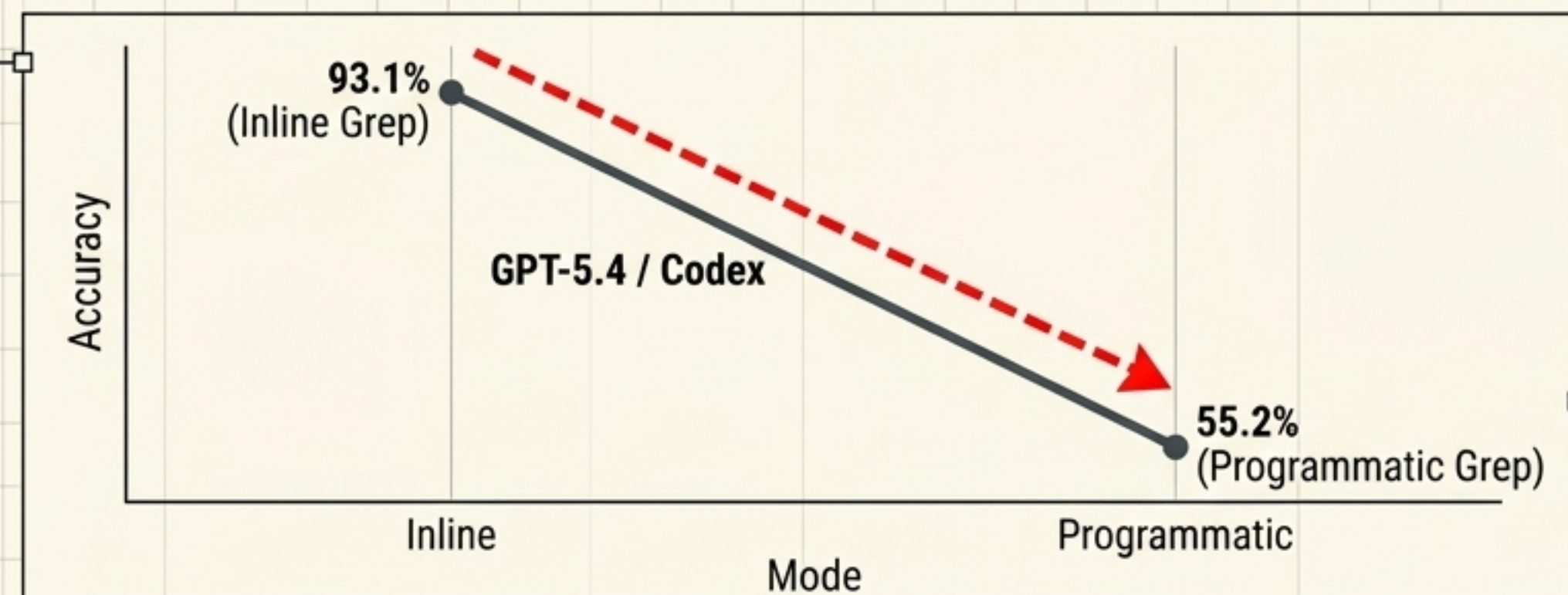
ผลลัพธ์ที่ 1: ชัยชนะของ Grep และ "บทลงโทษ" จาก Programmatic



Insight 1:

Grep ชนะเสมอในแบบ Inline:

การค้นหาแบบตรงตัว (Lexical)
ดึงข้อมูลจำเพาะ (Literal spans)
ได้แม่นยำกว่าในทุก Harness

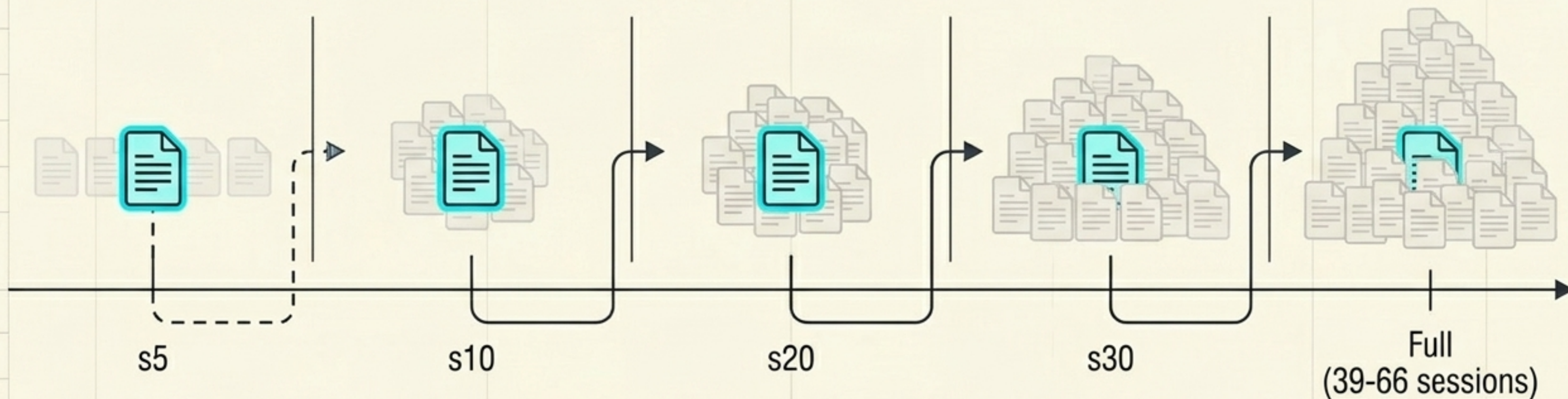


Insight 2:

ระบบรวมเมื่อเปลี่ยนเป็น Programmatic:

การบังคับให้ LLM ดึงข้อมูลจากไฟล์เอง
ทำให้ประสิทธิภาพตกฮวบ (เช่น GPT-5.4
ความแม่นยำร่วงจาก 93.1% เหลือ 55.2%)

สังเวียนทดสอบที่ 2: เมื่อสัญญาณรบกวน (Noise) เพิ่มขึ้น



เป้าหมาย

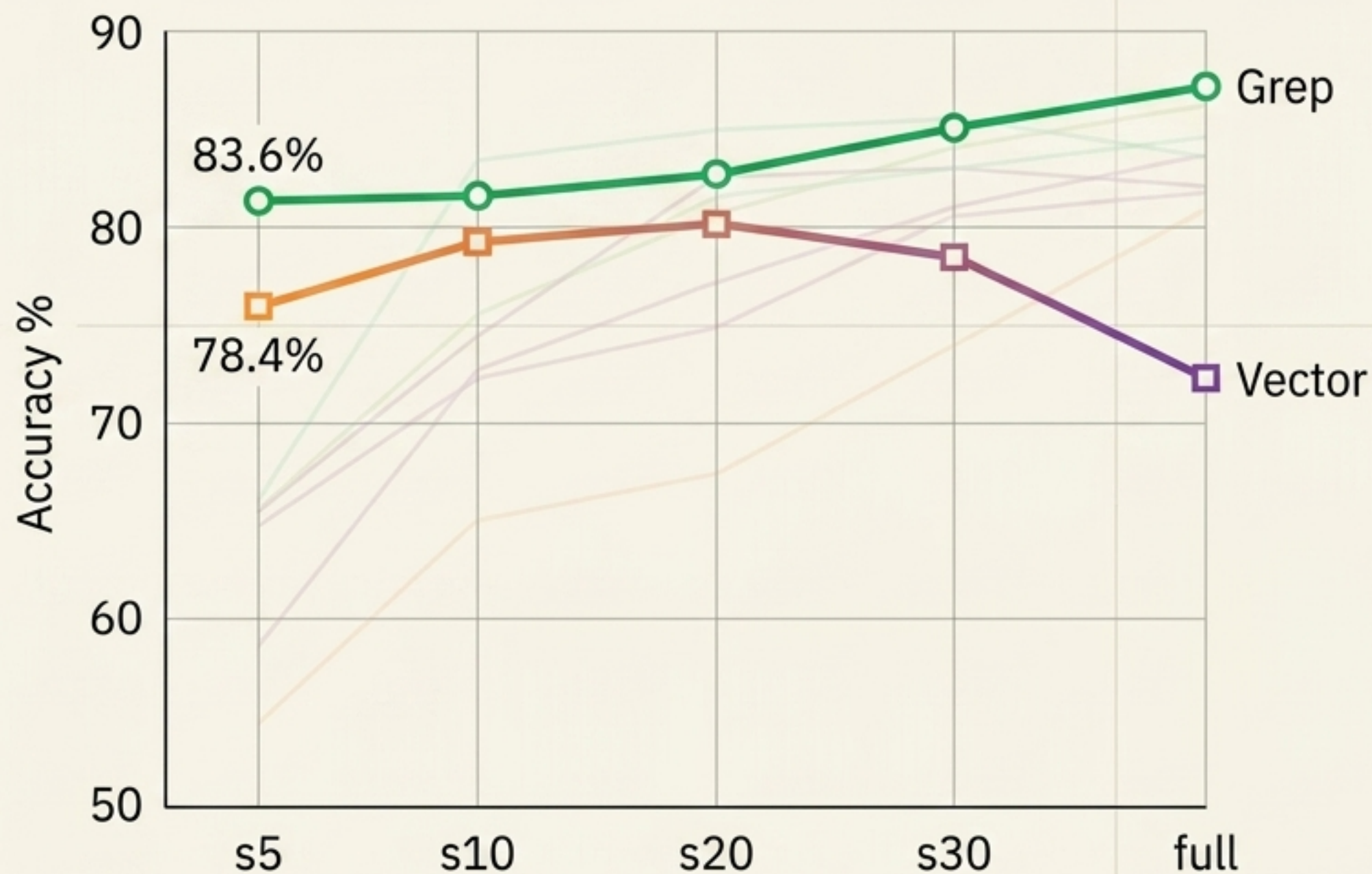
ทดสอบความทนทานของ Grep และ Vector เมื่อสัดส่วนข้อมูลขยะต่อข้อมูลที่เกี่ยวข้องสูงขึ้น

วิธีการ

เพิ่มประวัติการสนทนาที่ไม่เกี่ยวข้อง (Distractor sessions) ทีละระดับ ตั้งแต่ 5 sessions ไปจนถึง Full Haystack

เมื่อข้อมูลขยายตัว Vector จะยังคงเป็นผู้ชนะตามความเชื่อเดิมหรือไม่?

ผลลัพธ์ที่ 2: จุดตัด (Crossover Effect) เมื่อข้อมูลขยายตัว



Insight 1

Vector ดีในช่วงต้น: เข้าถึงข้อมูลเชิงบริบทได้ดีเมื่อขนาดข้อมูลยังเล็ก

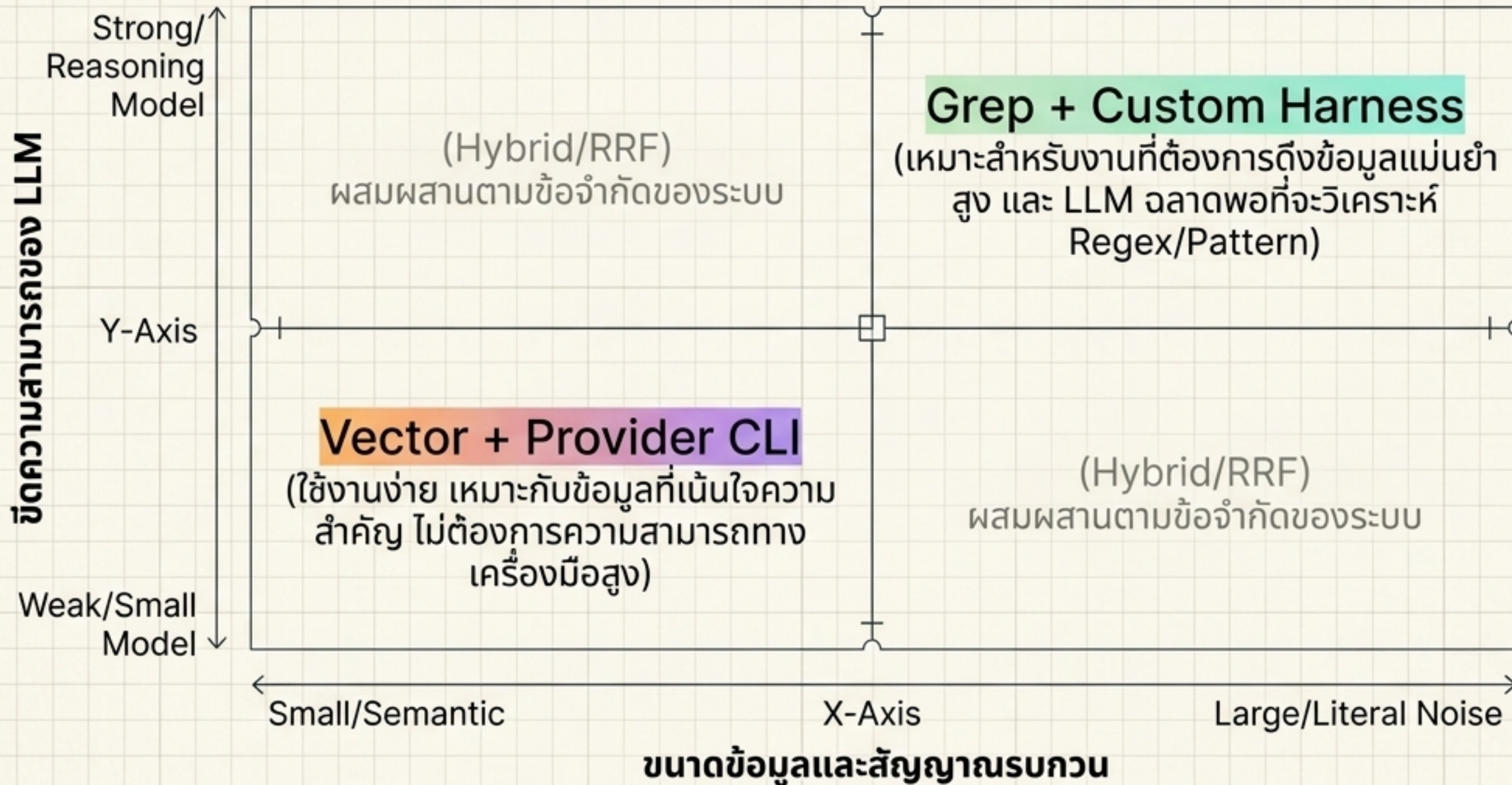
Insight 2

Grep แม่นยำเมื่อข้อมูลมหาศาล: ทนทานต่อ "คำหลอก" (False friends) ได้ดีกว่าเมื่อต้องเผชิญกับ Noise ระดับสูง

Critical Nuance

ความทนทานต่อสเกลไม่ได้อยู่ที่ตัวค้นหาเพียงอย่างเดียว แต่อยู่ที่ว่า Agent Harness ตัวใดเป็นผู้ควบคุม (เช่น Claude Code เอื้อประโยชน์ให้กับ Grep เสมอ)

สถาปัตยกรรมการตัดสินใจ (Architectural Decision Matrix)



บทสรุปสำหรับนักพัฒนา AI (Core Takeaways)

